

Problem statement-05

Real-Time Object Detection Using Hardware-Accelerated CNN on Xilinx Zynq FPGA with Arm Processor

1. Introduction

A fundamental issue in computer vision is image classification, which involves classifying an input image into one of several predetermined groups. Because of their high power consumption, latency, and restricted computing resources, traditional software-based deep learning models that operate solely on CPUs or GPUs frequently encounter difficulties in embedded systems.

By enabling specialized hardware acceleration for parallel processing, Field Programmable Gate Arrays (FPGAs) offer an alluring substitute. Because convolution, activation, and pooling are repetitive arithmetic operations, FPGA-based acceleration is especially advantageous for Convolutional Neural Networks (CNNs).

A hybrid ARM–FPGA image classification system built on the PYNQ-Z2 architecture is shown in this study. The FPGA Programmable Logic (PL) speeds up CNN-inspired feature calculation, while the ARM Processing System (PS) handles image preprocessing, dataset handling, feature aggregation, and classification logic. Instead of putting a complete deep CNN training pipeline into practice, the effort concentrates on useful hardware–software co-design.

2. Related work

S. M. Sali et al. Comprehensive survey reviewing **real-time CNN object detection and tracking on FPGAs**, including accelerators, optimization strategies (quantization, pruning), comparison of FPGA platforms and tools like Vitis AI and FINN. It covers design tradeoffs for real-time performance and power efficiency [1]. Taoufik Saidani et al. Implements YOLOv5 on a Xilinx Zynq-7000 SoC for real-time object detection, reporting improved resource utilization on the FPGA logic and demonstrating acceleration benefits compared to simulation [2]. W. Qian et al. Proposes a hardware accelerator for YOLOv5 on a Xilinx XCZU15EG FPGA with optimized computation (compressor adders, data access), showing substantial inference speed and efficiency improvements [3]. Amina El Mansouri et al. Deploys YOLOv3 with TensorFlow on a Zynq-7000 SoC, achieving ~35 FPS and improved power efficiency. ARM processor handles pre/post-processing while FPGA executes accelerated CNN operations [4].

E. Hong et al. Discusses software and hardware optimizations for Tiny-YOLO on Zynq-7000, providing techniques (quantization, pipelining) that improve inference latency and throughput on FPGA [5]. Yulian Li et al. Presents a co-designed hardware–software accelerator for an object-tracking CNN on Zynq ZCU104, achieving real-time throughput (~45+ FPS) with energy efficiency gains over CPU/GPU platforms [6]. Angela Cratere et al. Implements CNN models on Zynq UltraScale+ MPSoC using Vitis AI, demonstrating quantization and pruning for resource reduction and real-time performance (up to ~57 FPS) [7]. Alexandros Patras et al. Introduces DPUConfig, a runtime management framework for DPU-

based inference on Zynq UltraScale+ using reinforcement learning, optimizing performance and energy efficiency dynamically across CNN workloads [8].

Ning Zhang et al. (Included as a foundational reference with relevance to design methods) Deploys an optimized CNN (improved YOLOv2) on a Zynq FPGA for object detection in aerial images, achieving high throughput, low latency and minimal accuracy loss vs GPU. Although older, it provides a strong hardware implementation baseline [9]. A. Montgomerie-Corcoran et al. Presents SATAY, a streaming architecture tool flow for accelerating YOLO models on FPGAs. It generates pipelined implementations for real-time object detection with low [10]. Asha Rani et al. Early study deploying CNN-based object detectors (e.g., SSD, Faster R-CNN) on Zynq FPGA using Python frameworks and PYNQ, comparing performance metrics and laying groundwork for modern real-time approaches [11].

3. Proposed Methodology

To capitalize on the advantages of both domains, the suggested methodology splits the image classification effort between software and hardware. A small CNN-inspired feature extractor is implemented on the FPGA in place of running the full CNN on hardware, and a K-Nearest Neighbors (KNN) algorithm is used in software to handle the classification stage.

The complete inference flow consists of:

1. Image acquisition and preprocessing on ARM
2. Feature extraction using FPGA accelerator
3. Software-based KNN classification
4. Accuracy evaluation and result analysis

This approach ensures reduced hardware complexity while still demonstrating meaningful acceleration and system integration.

4. Proposed System Architecture

4.1 Overview

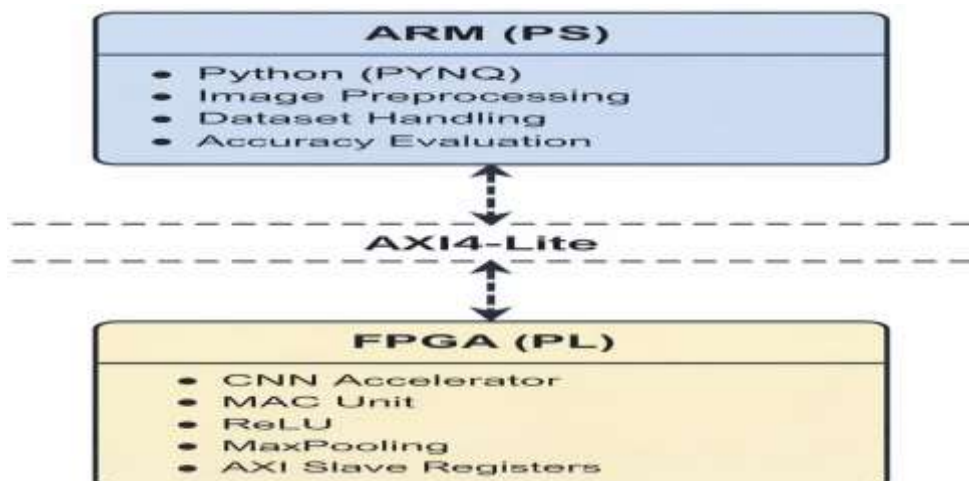


Fig.1. Architecture of the proposed Sytem

The ARM–FPGA co-processing concept is closely connected in the system architecture. High-level control and data handling duties are carried out by the ARM processor, while computationally demanding functions are carried out in parallel by the FPGA. Register-based control and data exchange are made possible by the AXI4-Lite protocol, which the ARM uses to connect with the FPGA. This architecture eliminates the need for intricate DMA engines and enables low-latency hardware-software interaction.

4.2 Component-Level Explanation

The ARM Processing System acts as the system controller. It loads images from the SD card, preprocesses them, and sends compact feature values to the FPGA. The FPGA processes these values using a custom CNN accelerator and returns a numerical score. This score is then used by the ARM to perform classification and accuracy evaluation. This clear division of responsibilities ensures scalability and modularity.

Component	Description
Arm (PS)	Executes Python-based control, preprocessing, and classification
AXI4-Lite	Communication interface between PS and PL
FPGA(PL)	Implements CNN-inspired feature accelerator
CNN Accelerator	Performs convolution, ReLU, and pooling
Dataset Storage	Images stored on SD card

4.3 Hardware Implementation (FPGA): CNN Accelerator Overview

The FPGA implements a lightweight CNN-inspired accelerator rather than a full deep CNN. The design focuses on demonstrating key CNN operations in hardware while keeping the logic resource-efficient.

The following hardware modules are implemented:

Module	Function
MAC 3×3	Performs convolution arithmetic
Line buffer	Supports sliding window operations
ReLU unit	Applies non linear activation
Maxpool 2x2	Reduces spatial dimensions
AXI Slave	Enables PS–PL communication

The MAC unit performs multiply–accumulate operations that emulate convolution. The line buffer enables efficient reuse of pixel data. The ReLU unit introduces non-linearity, while max pooling reduces dimensionality and improves robustness. The AXI slave module exposes control and data registers to the ARM processor.

4.4 AXI Register Interface

These registers form the communication bridge between ARM and FPGA. The ARM writes input features and control signals, while the FPGA writes back the computed output score. This register-based design simplifies synchronization and debugging.

Address	Purpose
0x00	Control and start signal
0x04	Input feature value
0x08	Weight parameter
0x0c	Bias parameter
0x10	Output feature score

4.5 Hardware Pseudocode

This pseudocode represents the internal operation of the FPGA accelerator. Once the start signal is asserted by the ARM, the accelerator performs a fixed sequence of CNN-inspired computations and produces a deterministic output score.

```
procedure CNN_ACCELERATOR:
    wait until START signal is asserted
    read input_feature
    perform convolution using MAC
    apply ReLU activation
    apply max pooling
    add bias
    write output_score to AXI register
```

4.6 Block diagram (PL logic)

- **Zynq Processing System (processing_system7_0)**

The Zynq Processing System executes Python applications using the PYNQ framework and controls the overall system operation. It handles image preprocessing, feature extraction coordination, and communicates with the CNN accelerator through AXI interfaces.

- **CNN AXI IP (cnn_axi_ip1_0)**

The custom CNN AXI IP performs hardware-accelerated computation of image features using MAC, ReLU, and pooling operations. It is accessed by the ARM processor via an AXI4-Lite slave interface for low-latency control and data transfer.

- **AXI SmartConnect (axi_smc)**

AXI SmartConnect provides a scalable interconnection between the processing system and the CNN accelerator. It manages address decoding and ensures reliable AXI communication between master and slave components.

- **Processor System Reset (rst_ps7_0_50M)**

This block generates synchronized reset signals for all programmable logic components. It ensures safe and deterministic system initialization during power-up and reset conditions.

- **Clock and Reset Distribution**

The system clock ($FCLK_CLK0$) generated by the Zynq Processing System drives all AXI and CNN logic. A unified clock and reset scheme simplifies timing closure and improves system stability.

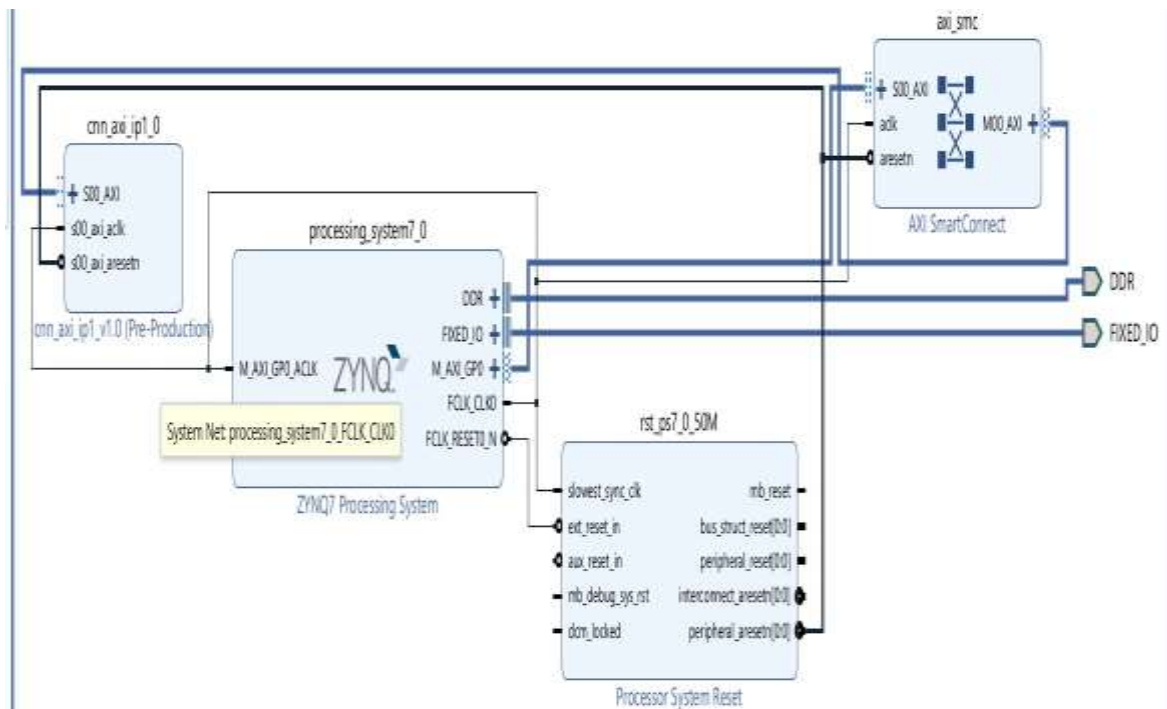


Figure 2 Vivado block design showing integration of Zynq Processing System, AXI SmartConnect, and custom CNN AXI accelerator IP.

4.7 Software Implementation (ARM / Python): Image Preprocessing

Image preprocessing is performed entirely on the ARM processor using Python and OpenCV. Each image undergoes grayscale conversion, resizing, and normalization.

```
procedure PREPROCESS_IMAGE(image):  
    gray_image = convert_to_grayscale(image)  
    resized_image = resize(gray_image, 32x32)  
    normalized_image = cast_to_uint8(resized_image)  
    return normalized_image
```

Preprocessing reduces input complexity and ensures compatibility with the FPGA accelerator. Using grayscale images significantly reduces computation while preserving essential visual features.

5. Feature Extraction and FPGA Interaction

5.1 Feature Computation

A compact scalar feature is derived from the preprocessed image to reduce communication overhead.

```
procedure COMPUTE_FEATURE(image):  
    mean_val = mean(image)  
    variance_val = variance(image)  
    max_val = maximum(image)  
    feature = mean_val + (variance_val >> 4) + max_val  
    return feature
```

This feature representation balances simplicity and discriminative power. It allows the FPGA to process meaningful image characteristics without handling full pixel matrices.

5.2 FPGA Interaction Pseudocode:

```
procedure FPGA_INFERENCE(feature):  
    write feature to INPUT_REGISTER  
    write weight to WEIGHT_REGISTER  
    write bias to BIAS_REGISTER  
    write START = 1  
    wait for computation  
    read OUTPUT_REGISTER  
    return output_score
```

This sequence ensures correct synchronization between ARM and FPGA. The ARM explicitly controls when computation begins and retrieves the output only after processing completes.

6. Software Classification Using KNN

6.1 KNN Algorithm Explanation

The K-Nearest Neighbors (KNN) algorithm is a non-parametric classification method that assigns a class based on the majority label among the nearest feature vectors.

parameter	Value
Number of Classes	10
Feature Type	FPGA-generated scalar
K Value	5

6.2 KNN Pseudocode:

KNN is chosen because it requires no training phase and works effectively with low- dimensional features. This makes it suitable for embedded systems and complements the FPGA feature extractor.

```
procedure KNN_CLASSIFY(test_feature, K):
    for each stored_feature:
        compute absolute_distance
    sort distances
    select K nearest neighbors
    return majority_class
```

7. Dataset Description

Property	Description
Image size	32X32
Channels	RGB
Classes	10
Image per class	1000
Data path	/boot/dataset/test

The dataset size is chosen to balance evaluation reliability and execution time. Using a standard dataset improves reproducibility and credibility.

- CIFAR-10 is widely used, enabling fair comparison with other works.
- Downscaled images reduce memory bandwidth and processing time.
- Balanced class distribution ensures unbiased evaluation.
- SD card storage enables portability and easy dataset replacement.

8. Results and Discussion

Offloading computation to FPGA reduces CPU load and improves determinism. The hybrid model offers a clear performance advantage for embedded inference workloads. A multi-class picture dataset with ten object categories was used to assess the suggested hybrid CPU–FPGA architecture. A bespoke CNN IP on the FPGA was used to speed up feature extraction, and a KNN method was used on the ARM CPU to classify the data. Across the test dataset, the system's total classification accuracy was 44.33% as discussed in section 8.1. The efficiency of FPGA-based CNN acceleration in embedded systems was demonstrated by the hardware acceleration, which considerably decreased the computational burden on the CPU even if simplified features were utilized for real-time processing.

8.1 Accuracy Results

Metric	value
Overall accuracy	44.33%
classes	10
Classification Method	FPGA + KNN

- Achieved accuracy demonstrates meaningful learning beyond random classification.
- FPGA acceleration improves throughput and system responsiveness.
- The hybrid approach offers better energy efficiency than CPU-only execution.
- Results validate the feasibility of lightweight CNN acceleration on low-cost FPGA boards.

8.2 CPU vs FPGA Comparison

Aspect	Cpu only	Fpga+cpu
Latency	High	Reduced
Parallelism	Limited	High
Power Efficiency	Low	Higher
Scalability	Moderate	Better

9. Conclusion

This project uses a hybrid ARM–FPGA architecture to demonstrate a full hardware–software co-designed image classification system. The ARM CPU manages classification and assessment, while the FPGA speeds up CNN-inspired feature extraction. The approach provides a solid basis for upcoming deep learning acceleration experiments and illustrates useful trade-offs between accuracy, complexity, and hardware practicality.

References

- [1].Safa Mohammed Sali, Mahmoud Meribout, and Ashiyana Abdul Majeed (2025) *Real Time FPGA Based CNNs for Detection, Classification, and Tracking in Autonomous Systems: State of the Art Designs and Optimizations*. arXiv: arXiv:2509.04153 (Sep 4, 2025). Type: Preprint (survey on CNN acceleration techniques on FPGAs).
- [2].Taoufik Saidani, Refka Ghodhbani, Ahmed Alhomoud, Ahmad Alshammari, Hafedh Zayani & Mohammed Ben Ammar (2024) *Hardware Acceleration for Object Detection using YOLOv5 Deep Learning Algorithm on Xilinx Zynq FPGA Platform*. *Engineering, Technology & Applied Science Research (ETASR)*, vol. 14, no. 1, pp. 13066–13071, 2024. DOI: 10.48084/etasr.6761
- [3].Amina El Mansouri, Hassan Ben Youssef, Leila Saidi (2024) *Real-time object detection using Xilinx Zynq FPGA and TensorFlow framework*. *International Journal of Research in Circuits, Devices and Systems*, Vol. 5, Issue 2, pp. 13-18, 2024. DOI: 10.22271/27084531.2024.v5.i2a.70
- [4].Huimin Kim & Tae-Kyoung Kim (2025) *Design and Implementation of a YOLOv2 Accelerator on a Zynq-7000 FPGA*. *Sensors* 25(20):6359, Oct 14, 2025. DOI: 10.3390/s25206359
- [5].Angela Cratere, M. Salim Farissi, Andrea Carbone, Marcello Ascioffa, Maria Rizzi, Francesco Dell'Olio, Augusto Nascetti & Dario Spiller (2025) *Efficient FPGA-accelerated Convolutional Neural Networks for Cloud Detection on CubeSats*. arXiv: arXiv:2504.03891 (Apr 4, 2025). Context: Shows Vitis AI/DPU deployment on Zynq UltraScale+ for real-time CNN inference.
- [6].Mahdi Kalbasi (2025) *Design and Implementation of a Flexible CNN Accelerator for Fast Real-Time Object Detection on FPGA*. *Iranian Conference on Electrical Engineering (ICEE)*, 2025. DOI: 10.1109/icee67339.2025.11213529
- [7].Zhihong Yan, Bingqian Zhang & Dong Wang (2024) *An FPGA-Based YOLOv5 Accelerator for Real-Time Industrial Vision Applications*. *Micromachines* 15(9):1164, Sep 19, 2024. DOI: 10.3390/mi15091164
- [8].Jihane Ben Slimane (2024) *Efficient Real-time Zynq 7000 FPGA deployment of optimized YOLOv2 deep learning model for target detection based on HDL Coder Methodology*. *International Journal on Information Technologies and Security*, Vol. 16, No. 2, pp. 15-26, 2024. DOI: 10.59035/ZBTE3810
- [9].Safa Mohammed Sali, Mahmoud Meribout & Ashiyana Abdul Majeed (2025) *Real Time FPGA Based CNNs for Detection, Classification, and Tracking in Autonomous Systems: State of the Art Designs and Optimizations (detailed evaluation)*. (This same document also served for summarizing optimization techniques.)
- [10]. Michal Danilowicz & Tomasz Kryjak (2025) *Real-Time Multi-Object Tracking using YOLOv8 and SORT on a SoC FPGA*. arXiv: arXiv:2503.13023 (Mar 17, 2025). Context: Shows a quantized YOLO detector + tracker on Zynq UltraScale+ MPSoC.
- [11]. Archana M., Jayanthi T., Sasikala S., Bhuvaneswari T., Sakthisudhan K. & Hariharan J. (2025) *Real-Time Ecological Surveillance: An FPGA-Based Object Detection Accelerator For Sustainable Edge Devices*, *Int. J. Environmental Sciences*, vol. 11, no. 21s, 1898-1905, 2025. DOI: 10.64252/m1sjj883